Cross-Venue Momentum Trading System

Implementation Specification for Cursor

Version 1.0

March 11, 2026

Abstract

This document specifies a trading system that monitors Uniswap V3 swap events across three fee-tier pools (30 bps, 5 bps, 1 bps) on Arbitrum, constructs directional momentum signals, and places maker limit orders on Deribit perpetual futures. Every component, data structure, function signature, formula, and database schema is defined here so that the system can be implemented directly from this specification.

Contents

1 System Overview **4**

2 Project Structure **4**

3 Dependencies **5**

4 Configuration **5**
4.1 config.yaml . . . 5
4.2 config.py Specification . . . 6

5 Data Models **6**
5.1 Enumerations . . . 7
5.2 SwapEvent . . . 7
5.3 SignalState . . . 7
5.4 OrderRequest, OrderState, Position, TradeRecord . . . 8

6 Database Schema **9**
6.1 Database Class Interface . . . 10

7 Event Listener **10**
7.1 Responsibility . . . 11
7.2 Swap Event Decoding . . . 11
7.3 Price Computation . . . 11
7.4 Log-Return Computation . . . 11
7.5 Class Interface . . . 11

8 Signal Engine **12**
8.1 Overview . . . 12
8.2 Per-Pair Internal State . . . 12
8.3 Layer 1: Trend Filter (30 bps Pool) . . . 13
8.3.1 Decay Parameter . . . 13

8.3.2 Conviction Update . . . 13
8.3.3 Intensity Update . . . 13
8.3.4 Trend State with Hysteresis . . . 13
8.4 Layer 2: Momentum Confirmation (5 bps Pool) . . . 13
8.4.1 Rolling Window . . . 13
8.4.2 Cumulative Momentum . . . 13
8.4.3 Lag-1 Autocorrelation . . . 14
8.4.4 Momentum Flag . . . 14
8.5 Combined Signal . . . 14
8.6 Transition Detection . . . 14
8.7 Class Interface . . . 14

9 Deribit Client **15**
9.1 Responsibility . . . 15
9.2 Authentication . . . 15
9.3 Subscriptions . . . 15
9.4 Order Placement . . . 16
9.5 Order Cancellation . . . 16
9.6 Position Query . . . 16
9.7 Class Interface . . . 16

10 Risk Manager **17**
10.1 Responsibility . . . 17
10.2 Pre-Trade Checks . . . 17
10.3 Stop-Loss Monitor . . . 18
10.4 Class Interface . . . 18

11 Execution Manager **19**
11.1 Responsibility . . . 19
11.2 Limit Price Computation . . . 19
11.3 Position Sizing . . . 19
11.4 Signal Transition Handlers . . . 19
11.4.1 On Entry . . . 19
11.4.2 On Exit . . . 19
11.4.3 On Reversal . . . 19
11.4.4 On Scale . . . 20
11.4.5 On Reduce . . . 20
11.5 Staleness Check . . . 20
11.6 Order Update Handler . . . 20
11.7 Forced Exit Handler . . . 20
11.8 Class Interface . . . 20

12 Main Entry Point **21**

13 Backtesting Framework **23**
13.1 Purpose . . . 23
13.2 Architecture . . . 23
13.3 Simulated Fill Logic . . . 23
13.4 BacktestResult . . . 24
13.5 Alpha Decay Curve . . . 24
13.6 Running the Backtest . . . 24

14 Logging **25**

15 Testing Requirements **25**
15.1 test_signal_engine.py 26
15.2 test_execution_manager.py 26
15.3 test_risk_manager.py 26

16 Deployment Phases **26**

17 Kill Criteria **27**

1 System Overview

The system has four runtime components connected in a linear pipeline:

1. **Event Listener** — subscribes to on-chain `Swap` events on three Uniswap V3 pools per token pair via WebSocket RPC.
2. **Signal Engine** — receives parsed events, maintains rolling statistics, and computes a combined momentum signal $S_t \in [-1, +1]$.
3. **Execution Manager** — translates signal transitions into Deribit limit-order lifecycle actions (place, reprice, cancel, exit).
4. **Risk Manager** — enforces position limits, stop losses, daily loss limits, margin constraints, and cooldown periods. It has veto authority over every order.

A **Database** module persists all events, signals, orders, and trades to SQLite. A **Deribit Client** module handles authentication and the REST/WebSocket API. The entire system runs as a single Python 3.11+ `asyncio` process.

Operating Modes

The system supports three modes selected via `config.yaml`: **live** (real orders on Deribit), **paper** (signal pipeline runs, orders are simulated), **backtest** (replay historical events from the database).

2 Project Structure

```
momentum_system/
    main.py                    # Entry point, wires components, runs
        event loop
    config.py                  # Load + validate config.yaml via
        Pydantic
    config.yaml                # User-editable configuration
    models.py                  # All data classes and enums
    event_listener.py          # On-chain Swap event ingestion
    signal_engine.py           # Signal computation
    deribit_client.py          # Deribit REST + WebSocket client
    execution_manager.py       # Order lifecycle
    risk_manager.py            # All risk checks
    database.py                # SQLite persistence (async SQLAlchemy)
    backtest.py                # Backtesting framework
    logger.py                  # Structured logging setup
    requirements.txt
    .env                       # DERIBIT_CLIENT_ID,
        DERIBIT_CLIENT_SECRET
    tests/
        test_signal_engine.py
        test_execution_manager.py
        test_risk_manager.py
        test_backtest.py
```

3 Dependencies

```
web3>=6.15.0
websockets>=12.0
aiohttp>=3.9.0
pyyaml>=6.0
pydantic>=2.5.0
numpy>=1.26.0
sqlalchemy>=2.0.0
aiosqlite>=0.19.0
python-dotenv>=1.0.0
pytest>=7.4.0
pytest-asyncio>=0.23.0
```

No other libraries should be used.

4 Configuration

4.1 config.yaml

```
system:
  log_level: "INFO"
  heartbeat_interval_seconds: 60
  database_path: "data/events.db"
  mode: "live"

infrastructure:
  rpc_url: "wss://arb-mainnet.g.alchemy.com/v2/YOUR_KEY"
  chain_id: 42161
  block_confirmations: 1
  reconnect_delay_seconds: 5
  max_reconnect_attempts: 50

deribit:
  base_url: "https://www.deribit.com"
  ws_url: "wss://www.deribit.com/ws/api/v2"
  client_id: "${DERIBIT_CLIENT_ID}"
  client_secret: "${DERIBIT_CLIENT_SECRET}"

pairs:
  - name: "ETH-USDC"
    deribit_instrument: "ETH-PERPETUAL"
    token0: "0x82aF49447D8a07e3bd95BD0d56f35241523fBab1"
    token1: "0xaf88d065e77c8cC2239327C5EDb3A432268e5831"
    token0_decimals: 18
    token1_decimals: 6
    invert_price: false

    pools:
      bp30:
        address: "0xc473e2aEE3441BF9240Be85eb122aBB059A3B57c"
        fee: 3000
      bp5:
        address: "0xC6962004f452bE9203591991D15f6b388e09E8D0"
        fee: 500
```

```yaml
    bp1:
      address: "0xd9e2a1a61B6E61b275cEc326465d417e52C1b95c"
      fee: 100

  signal:
    conviction_halflife_seconds: 1800
    trend_entry_threshold: 1.5
    trend_exit_threshold: 0.5
    intensity_smoothing: 0.9
    momentum_window_events: 50
    min_autocorrelation: 0.05
    conviction_cap: 3.0

  execution:
    offset_base_bps: 2.0
    offset_conviction_bps: 5.0
    stale_order_seconds: 60
    allow_reprice: false
    max_reprice_bps: 3.0
    post_only: true

  risk:
    max_position_usd: 50000
    max_position_contracts: 100
    stop_loss_bps: 20
    take_profit_bps: 30
    cooldown_seconds: 300
    daily_loss_limit_usd: 500
    max_open_orders: 3
    margin_usage_limit_pct: 50
```

4.2 config.py Specification

Implement a function `load_config(path: str = "config.yaml") -> Config` that:

(a) Reads the YAML file.

(b) Walks the resulting dictionary and replaces any string matching `${VAR_NAME}` with `os.environ["VAR_NAM` Raises `ValueError` if the variable is unset.

(c) Parses the dictionary into nested Pydantic models: `SystemConfig`, `InfraConfig`, `DeribitConfig`, and a list of `PairConfig` (each containing `PoolsConfig`, `SignalConfig`, `ExecutionConfig`, `RiskConfig`).

(d) Returns a root `Config` model that holds all sub-models.

Pydantic validation should enforce: all threshold values are positive, `trend_exit_threshold < trend_entry_threshold`, `stop_loss_bps > 0`, `margin_usage_limit_pct` is between 1 and 100.

5 Data Models

All data classes live in `models.py` and use `pydantic.BaseModel`.

5.1 Enumerations

```
class FeeTier(str, Enum):
    BP1 = "bp1"
    BP5 = "bp5"
    BP30 = "bp30"

class Direction(int, Enum):
    LONG = 1
    SHORT = -1
    NEUTRAL = 0

class SignalTransition(str, Enum):
    ENTRY = "entry"
    EXIT = "exit"
    SCALE = "scale"
    REDUCE = "reduce"
    REVERSAL = "reversal"
    NONE = "none"

class OrderStatus(str, Enum):
    PENDING = "pending"
    PLACED = "placed"
    PARTIALLY_FILLED = "partially_filled"
    FILLED = "filled"
    CANCELLED = "cancelled"
    REJECTED = "rejected"
```

5.2 SwapEvent

```
class SwapEvent(BaseModel):
    pair_name: str
    fee_tier: FeeTier
    block_number: int
    block_timestamp: int          # unix seconds
    transaction_hash: str
    pool_address: str
    sqrt_price_x96: int
    tick: int
    liquidity: int
    amount0: int
    amount1: int
    price: float                  # human-readable, in quote/base
        terms
    log_return: Optional[float]   # None for first event per pool
    direction: Optional[int]      # sign(log_return)
```

5.3 SignalState

```
class SignalState(BaseModel):
    pair_name: str
    timestamp: float
    conviction_30: float          # C^{30}_t
```

```
    trend_state_30: int             # T^{30}_t in {-1, 0, +1}
    momentum_5: float               # M^{5}_t
    autocorrelation_5: float        # rho^{5}_t
    momentum_flag_5: bool           # F^{5}_t
    combined_signal: float          # S_t in [-1, +1]
    transition: SignalTransition
    intensity_30: float             # events per second
```

5.4 OrderRequest, OrderState, Position, TradeRecord

```
class OrderRequest(BaseModel):
    pair_name: str
    instrument: str                 # e.g. "ETH-PERPETUAL"
    direction: Direction
    size: float                     # in contracts
    limit_price: float
    post_only: bool = True
    reduce_only: bool = False
    label: str = ""

class OrderState(BaseModel):
    order_id: str
    request: OrderRequest
    status: OrderStatus
    placed_at: float
    filled_at: Optional[float] = None
    fill_price: Optional[float] = None
    filled_size: float = 0.0
    cancel_reason: Optional[str] = None

class Position(BaseModel):
    pair_name: str
    instrument: str
    direction: Direction
    size: float                     # absolute, in contracts
    entry_price: float
    entry_time: float
    unrealized_pnl: float = 0.0
    signal_at_entry: float = 0.0

class TradeRecord(BaseModel):
    pair_name: str
    instrument: str
    direction: Direction
    entry_price: float
    exit_price: float
    size: float
    entry_time: float
    exit_time: float
    gross_pnl_usd: float
    fees_usd: float
    net_pnl_usd: float
    signal_at_entry: float
    exit_reason: str                # "take_profit", "stop_loss",
                                    # "signal_exit", "signal_reversal"
```

6 Database Schema

Use SQLAlchemy 2.0 async engine with `aiosqlite`. All tables are created on startup via `Database.initialize()`.

```sql
CREATE TABLE IF NOT EXISTS swap_events (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    pair_name         TEXT    NOT NULL,
    fee_tier          TEXT    NOT NULL,
    block_number      INTEGER NOT NULL,
    block_timestamp   INTEGER NOT NULL,
    transaction_hash  TEXT    NOT NULL,
    pool_address      TEXT    NOT NULL,
    sqrt_price_x96    TEXT    NOT NULL,
    tick              INTEGER NOT NULL,
    liquidity         TEXT    NOT NULL,
    amount0           TEXT    NOT NULL,
    amount1           TEXT    NOT NULL,
    price             REAL    NOT NULL,
    log_return        REAL,
    direction         INTEGER,
    created_at        REAL    NOT NULL,
    UNIQUE(transaction_hash, pool_address)
);

CREATE TABLE IF NOT EXISTS signal_log (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    pair_name         TEXT    NOT NULL,
    timestamp         REAL    NOT NULL,
    conviction_30     REAL    NOT NULL,
    trend_state_30    INTEGER NOT NULL,
    momentum_5        REAL    NOT NULL,
    autocorrelation_5 REAL    NOT NULL,
    momentum_flag_5   INTEGER NOT NULL,
    combined_signal   REAL    NOT NULL,
    transition        TEXT    NOT NULL,
    intensity_30      REAL    NOT NULL,
    created_at        REAL    NOT NULL
);

CREATE TABLE IF NOT EXISTS order_log (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id          TEXT    NOT NULL,
    pair_name         TEXT    NOT NULL,
    instrument        TEXT    NOT NULL,
    direction         INTEGER NOT NULL,
    size              REAL    NOT NULL,
    limit_price       REAL    NOT NULL,
    status            TEXT    NOT NULL,
    placed_at         REAL    NOT NULL,
    filled_at         REAL,
    fill_price        REAL,
    filled_size       REAL    NOT NULL DEFAULT 0,
    cancel_reason     TEXT,
    label             TEXT,
    created_at        REAL    NOT NULL
);
```

```sql
CREATE TABLE IF NOT EXISTS trade_log (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    pair_name         TEXT    NOT NULL,
    instrument        TEXT    NOT NULL,
    direction         INTEGER NOT NULL,
    entry_price       REAL    NOT NULL,
    exit_price        REAL    NOT NULL,
    size              REAL    NOT NULL,
    entry_time        REAL    NOT NULL,
    exit_time         REAL    NOT NULL,
    gross_pnl_usd    REAL    NOT NULL,
    fees_usd          REAL    NOT NULL,
    net_pnl_usd      REAL    NOT NULL,
    signal_at_entry   REAL    NOT NULL,
    exit_reason       TEXT    NOT NULL,
    created_at        REAL    NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_swap_pair_tier
    ON swap_events(pair_name, fee_tier, block_timestamp);
CREATE INDEX IF NOT EXISTS idx_signal_pair
    ON signal_log(pair_name, timestamp);
CREATE INDEX IF NOT EXISTS idx_trade_pair
    ON trade_log(pair_name, exit_time);
```

6.1 Database Class Interface

```python
class Database:
    def __init__(self, db_path: str): ...
    async def initialize(self) -> None: ...
    async def insert_swap_event(self, event: SwapEvent) -> None: ...
    async def insert_signal_state(self, state: SignalState) -> None:
        ...
    async def insert_order(self, order: OrderState) -> None: ...
    async def update_order(self, order_id: str, **kwargs) -> None:
        ...
    async def insert_trade(self, trade: TradeRecord) -> None: ...
    async def get_swap_events(
        self, pair_name: str, fee_tier: FeeTier,
        since_timestamp: Optional[int] = None,
        limit: int = 500
    ) -> list[SwapEvent]: ...
    async def get_daily_pnl(
        self, pair_name: str, since_hours: int = 24
    ) -> float: ...
    async def get_recent_trades(
        self, pair_name: str, limit: int = 100
    ) -> list[TradeRecord]: ...
```

7 Event Listener

7.1 Responsibility

Subscribe to `Swap` events on every configured pool address via WebSocket RPC (`eth_subscribe` for logs). Parse each raw log into a `SwapEvent`, compute the human-readable price and log-return, persist to the database, and forward to the Signal Engine via an async callback.

7.2 Swap Event Decoding

The Uniswap V3 `Swap` event has the following ABI:

```
# topic0 = keccak256("Swap(address,address,int256,int256,
#                        uint160,uint128,int24)")
# = 0xc42079f94a6350d7e6235f29174924f928cc2ac818eb64
#     fed8004e115fbcca67
#
# Indexed: sender (topic1), recipient (topic2)
# Data:    amount0 (int256), amount1 (int256),
#          sqrtPriceX96 (uint160), liquidity (uint128),
#          tick (int24)
```

7.3 Price Computation

From `sqrtPriceX96`, compute the raw pool price:

$$P_{\text{raw}} = \left(\frac{\texttt{sqrtPriceX96}}{2^{96}}\right)^2 \tag{1}$$

Adjust for token decimals:

$$P = P_{\text{raw}} \times 10^{(\texttt{token0_decimals} - \texttt{token1_decimals})} \tag{2}$$

If `invert_price` is `true`, set $P \leftarrow 1/P$. This yields a price in quote-per-base terms (e.g. USD per ETH).

7.4 Log-Return Computation

Maintain a dictionary `last_price: dict[str, float]` keyed by pool address. For each event:

$$r = \begin{cases} \ln P_k - \ln P_{k-1} & \text{if a previous price exists for this pool} \\ \texttt{None} & \text{otherwise (first event)} \end{cases} \tag{3}$$

Set `direction = sign(r)` when r is not `None`.

7.5 Class Interface

```python
class EventListener:
    def __init__(
        self,
        config: Config,
        database: Database,
        on_event: Callable[[SwapEvent], Awaitable[None]]
    ): ...

    async def start(self) -> None:
        """Connect, subscribe to all pools, enter processing loop."""
```

```python
    async def stop(self) -> None:
        """Cancel tasks, close WebSocket."""

    def _parse_swap_log(
        self, log: dict,
        pair_name: str,
        fee_tier: FeeTier,
        pool_address: str,
        pair_config: PairConfig
    ) -> SwapEvent:
        """Decode log, compute price and log-return."""

    async def _handle_reconnect(self) -> None:
        """Wait reconnect_delay, re-connect, re-subscribe.
           Raise after max_reconnect_attempts."""
```

Implementation Notes

- Use `web3.py` `AsyncWeb3` with `WebSocketProvider`.
- Persist every event before forwarding to the signal engine.
- Events with `log_return = None` must still be forwarded; the signal engine handles them.
- On WebSocket disconnect, log a warning and invoke `_handle_reconnect`.

8 Signal Engine

8.1 Overview

The Signal Engine maintains per-pair state and computes two signal layers plus a combined trading signal. It receives every `SwapEvent` from the Event Listener and emits a `SignalState` to the Execution Manager whenever a signal transition occurs.

8.2 Per-Pair Internal State

For each configured pair, maintain the following mutable state:

```python
@dataclass
class PairSignalState:
    # Layer 1: 30 bps trend filter
    conviction_30: float = 0.0
    trend_state_30: int = 0          # {-1, 0, +1}
    last_bp30_timestamp: float = 0.0
    avg_bp30_delta: float = 300.0    # initial guess: 5 min
    last_conviction_update: float = 0.0

    # Layer 2: 5 bps momentum
    bp5_returns: deque               # maxlen = momentum_window_events
    bp5_prices: deque                # maxlen = momentum_window_events
        + 1

    # Combined
    previous_signal: float = 0.0
    previous_transition: SignalTransition = SignalTransition.NONE
```

Initialize `bp5_returns` and `bp5_prices` as empty deques with `maxlen` set to the configured window size.

8.3 Layer 1: Trend Filter (30 bps Pool)

Triggered on every `SwapEvent` with `fee_tier == BP30` and `log_return is not None`.

8.3.1 Decay Parameter

Compute the decay rate from the configured half-life:

$$\alpha_{30} = \frac{\ln 2}{T^{30}_{\text{half}}} \tag{4}$$

where T^{30}_{half} is `conviction_halflife_seconds`.

8.3.2 Conviction Update

Let t be the current event timestamp, t_{prev} be `last_conviction_update`, and $d_k = \text{sign}(r^{30}_k)$:

$$C^{30}_t = C^{30}_{t_{\text{prev}}} \cdot \exp\big(-\alpha_{30}(t - t_{\text{prev}})\big) + d_k \tag{5}$$

$$C^{30}_t \leftarrow \text{clamp}\Big(C^{30}_t,\ -C_{\max},\ C_{\max}\Big) \tag{6}$$

where $C_{\max}$ is `conviction_cap`. Update `last_conviction_update` $\leftarrow t$.

8.3.3 Intensity Update

$$\bar{\delta}^{30}_k = \beta \cdot \bar{\delta}^{30}_{k-1} + (1-\beta) \cdot (t - t_{\text{prev,bp30}}) \tag{7}$$

where β is `intensity_smoothing`. Then $\lambda^{30}_t = 1 \,/\, \bar{\delta}^{30}_k$. Update `last_bp30_timestamp` $\leftarrow t$.

8.3.4 Trend State with Hysteresis

Let $\theta_e =$ `trend_entry_threshold` and $\theta_x =$ `trend_exit_threshold`:

$$T^{30}_t = \begin{cases} +1 & \text{if } C^{30}_t > \theta_e \\ -1 & \text{if } C^{30}_t < -\theta_e \\ 0 & \text{if } |C^{30}_t| < \theta_x \\ T^{30}_{t^-} & \text{otherwise} \end{cases} \tag{8}$$

8.4 Layer 2: Momentum Confirmation (5 bps Pool)

Triggered on every `SwapEvent` with `fee_tier == BP5` and `log_return is not None`.

8.4.1 Rolling Window

Append the log-return to `bp5_returns`. The deque's `maxlen` automatically evicts the oldest entry when full.

8.4.2 Cumulative Momentum

$$M^5_t = \sum_{r \in \texttt{bp5_returns}} r \tag{9}$$

8.4.3 Lag-1 Autocorrelation

Given the returns vector $\mathbf{r} = (r_1, \ldots, r_N)$ in the deque, compute:

$$\bar{r} = \frac{1}{N} \sum_{i=1}^{N} r_i \tag{10}$$

$$\hat{\rho}_t^5 = \frac{\sum_{i=2}^{N} (r_i - \bar{r})(r_{i-1} - \bar{r})}{\sum_{i=1}^{N} (r_i - \bar{r})^2} \tag{11}$$

If $N < 5$ or the denominator is zero, set $\hat{\rho}_t^5 = 0$.

8.4.4 Momentum Flag

$$F_t^5 = \mathbb{1}[\text{sign}(M_t^5) = T_t^{30}] \cdot \mathbb{1}[\hat{\rho}_t^5 > \rho_{\min}] \tag{12}$$

where $\rho_{\min}$ is `min_autocorrelation`. Both conditions must hold.

8.5 Combined Signal

Recomputed after *every* event (from any tier):

$$S_t = T_t^{30} \cdot F_t^5 \cdot \min\left(\frac{|C_t^{30}|}{C_{\max}}, 1\right) \tag{13}$$

This yields $S_t \in [-1, +1]$. The sign is direction; the magnitude is conviction.

8.6 Transition Detection

Compare S_t to the previous signal S_{t^-}:

Condition	Transition	Action
$S_{t^-} = 0$ and $S_t \neq 0$	ENTRY	Place new order
$S_{t^-} \neq 0$ and $S_t = 0$	EXIT	Cancel open orders, close position
$\text{sign}(S_t) = \text{sign}(S_{t^-})$ and $\lvert S_t \rvert > \lvert S_{t^-} \rvert$	SCALE	Increase order size
$\text{sign}(S_t) = \text{sign}(S_{t^-})$ and $\lvert S_t \rvert < \lvert S_{t^-} \rvert$	REDUCE	Optional trim
$\text{sign}(S_t) \neq \text{sign}(S_{t^-})$ and both $\neq 0$	REVERSAL	Close and re-enter
Otherwise	NONE	No action

Emit a `SignalState` to the Execution Manager only when the transition is not NONE. Also persist every emitted `SignalState` to `signal_log`.

8.7 Class Interface

```python
class SignalEngine:
    def __init__(
        self,
        config: Config,
        database: Database,
        on_signal: Callable[[SignalState], Awaitable[None]]
    ): ...

    async def on_event(self, event: SwapEvent) -> None:
        """Route event to tier handler, recompute signal,
           detect transition, emit if non-trivial."""
```

```python
    def _process_bp30(self, pair: str, event: SwapEvent) -> None:
        """Update conviction, intensity, trend state."""

    def _process_bp5(self, pair: str, event: SwapEvent) -> None:
        """Update rolling returns, momentum, autocorrelation."""

    def _process_bp1(self, pair: str, event: SwapEvent) -> None:
        """Store latest 1bps price (used for reference only)."""

    def _compute_combined(self, pair: str) -> float:
        """Return S_t."""

    def _detect_transition(
        self, pair: str, new_signal: float
    ) -> SignalTransition:
        """Compare to previous signal, return transition type."""

    def get_state(self, pair: str) -> PairSignalState:
        """Return current internal state for a pair
           (used by risk manager and for logging)."""
```

9 Deribit Client

9.1 Responsibility

Handle all communication with Deribit: authentication, REST API calls, WebSocket subscriptions for order book data and private order updates.

9.2 Authentication

Deribit uses OAuth-style authentication. On connection, call:

```python
# WebSocket JSON-RPC request:
{
    "method": "public/auth",
    "params": {
        "grant_type": "client_credentials",
        "client_id": "<client_id>",
        "client_secret": "<client_secret>"
    }
}
```

This returns an `access_token` and `refresh_token`. Store them. Set a timer to call `public/auth` with `grant_type: refresh_token` before the access token expires (returned as `expires_in` seconds).

9.3 Subscriptions

After authentication, subscribe to:

(a) **Order book:** `book.{instrument}.100ms` — provides best bid/ask and mid price at 100 ms intervals.

(b) **User orders:** `user.orders.{instrument}.raw` — real-time updates on order status changes (fills, cancellations).

(c) **User trades:** `user.trades.{instrument}.raw` — confirmation of executed trades with fill prices.

(d) **Portfolio:** `user.portfolio.{currency}` — margin, equity, available funds.

9.4 Order Placement

Use the `private/buy` and `private/sell` JSON-RPC methods:

```
{
    "method": "private/buy",  # or "private/sell"
    "params": {
        "instrument_name": "ETH-PERPETUAL",
        "amount": 10,              # in contracts
        "type": "limit",
        "price": 3450.5,
        "post_only": true,
        "reduce_only": false,
        "label": "sig_entry_001"
    }
}
```

9.5 Order Cancellation

Use `private/cancel` with the `order_id` returned from placement.

For cancelling all open orders on an instrument: `private/cancel_all_by_instrument`.

9.6 Position Query

Use `private/get_position` with `instrument_name` to get current position size, direction, average entry price, and unrealized PnL.

9.7 Class Interface

```
class DeribitClient:
    def __init__(self, config: DeribitConfig): ...

    async def connect(self) -> None:
        """Open WebSocket, authenticate, start heartbeat."""

    async def disconnect(self) -> None:
        """Close WebSocket gracefully."""

    async def subscribe_book(
        self, instrument: str,
        callback: Callable[[dict], Awaitable[None]]
    ) -> None: ...

    async def subscribe_orders(
        self, instrument: str,
        callback: Callable[[dict], Awaitable[None]]
    ) -> None: ...

    async def subscribe_trades(
        self, instrument: str,
```

```
        callback: Callable[[dict], Awaitable[None]]
    ) -> None: ...

    async def subscribe_portfolio(
        self, currency: str,
        callback: Callable[[dict], Awaitable[None]]
    ) -> None: ...

    async def place_order(
        self, request: OrderRequest
    ) -> str:
        """Returns order_id."""

    async def cancel_order(self, order_id: str) -> None: ...

    async def cancel_all(self, instrument: str) -> None: ...

    async def get_position(
        self, instrument: str
    ) -> Optional[dict]: ...

    async def get_account_summary(
        self, currency: str
    ) -> dict: ...

    def get_mid_price(self, instrument: str) -> Optional[float]:
        """Return latest mid price from cached book data.
           Returns None if no book data received yet."""

    def get_best_bid(self, instrument: str) -> Optional[float]: ...
    def get_best_ask(self, instrument: str) -> Optional[float]: ...
```

WebSocket Management

The Deribit client must handle disconnections gracefully. If the WebSocket drops, log a warning, wait 5 seconds, reconnect, re-authenticate, and re-subscribe to all active subscriptions. Maintain a set of active subscriptions to enable this.

10 Risk Manager

10.1 Responsibility

The Risk Manager is consulted before every order placement. It has veto authority. It also runs a periodic background check for stop-loss conditions.

10.2 Pre-Trade Checks

The method `approve_order(request: OrderRequest) -> tuple[bool, str]` returns `(True, "")` if approved, or `(False, reason)` if rejected. It checks the following in order:

(1) **Cooldown.** If the pair is in cooldown (a stop-loss was triggered within the last `cooldown_seconds`), reject.

(2) **Daily loss limit.** Query `database.get_daily_pnl(pair_name)`. If cumulative losses exceed `daily_loss_limit_usd`, reject.

(3) **Position limit.** If the current position size plus the order size exceeds `max_position_contracts`, reject. If the notional value (size × price) exceeds `max_position_usd`, reject.

(4) **Open order limit.** If the number of open (unfilled, uncancelled) orders for this pair exceeds `max_open_orders`, reject.

(5) **Margin check.** Query the Deribit account summary. Compute the margin that would be consumed by the new position. If total margin usage would exceed `margin_usage_limit_pct` of equity, reject.

10.3 Stop-Loss Monitor

Run an async loop that checks every 1 second:

(1) For each open position, get the current Deribit mid price.

(2) Compute unrealized PnL in bps:

$$\text{pnl_bps} = \text{direction} \times \frac{P_{\text{now}} - P_{\text{entry}}}{P_{\text{entry}}} \times 10000 \tag{14}$$

(3) If pnl_bps $< -$`stop_loss_bps`, emit an immediate market exit (taker order via `private/buy` or `private/sell` with `type: market`).

(4) If pnl_bps $>$ +`take_profit_bps`, place a limit exit at the current mid (maker). If it doesn't fill within 5 seconds, convert to market.

(5) After a stop-loss exit, record the cooldown start time for the pair.

10.4 Class Interface

```python
class RiskManager:
    def __init__(
        self,
        config: Config,
        database: Database,
        deribit: DeribitClient
    ): ...

    async def approve_order(
        self, request: OrderRequest
    ) -> tuple[bool, str]: ...

    async def start_monitor(self) -> None:
        """Start the stop-loss/take-profit monitoring loop."""

    async def stop_monitor(self) -> None: ...

    def set_on_exit(
        self,
        callback: Callable[[str, str], Awaitable[None]]
    ) -> None:
        """Register callback(pair_name, exit_reason) for forced exits
            ."""

    def is_in_cooldown(self, pair_name: str) -> bool: ...

    def record_cooldown(self, pair_name: str) -> None: ...
```

11 Execution Manager

11.1 Responsibility

Translate signal transitions into Deribit order actions. Manage the full lifecycle of each order: placement, staleness cancellation, optional repricing, and exit orders for open positions.

11.2 Limit Price Computation

When placing an entry order, compute the limit price offset from the current Deribit mid price P_{mid}:

$$\Delta_t = \Delta_{\text{base}} + \Delta_{\text{conv}} \times (1 - |S_t|) \tag{15}$$

where Δ_{base} is `offset_base_bps` and Δ_{conv} is `offset_conviction_bps`, both in basis points. Convert to a price offset:

$$\delta_t = P_{\text{mid}} \times \frac{\Delta_t}{10000} \tag{16}$$

The limit price for a long entry is $P_{\text{mid}} - \delta_t$; for a short entry, $P_{\text{mid}} + \delta_t$.

Round the limit price to the instrument's tick size (0.5 for ETH-PERPETUAL, 0.5 for BTC-PERPETUAL on Deribit).

11.3 Position Sizing

Base size is `max_position_contracts`. Scale by signal magnitude:

$$\text{size} = \text{round}\big(|S_t| \times \texttt{max_position_contracts}\big) \tag{17}$$

Minimum size is 1 contract. The Risk Manager may further reduce this.

11.4 Signal Transition Handlers

11.4.1 On Entry

1. Compute limit price and size.
2. Build `OrderRequest` with `post_only=True`.
3. Call `risk_manager.approve_order(request)`. If rejected, log and return.
4. Call `deribit.place_order(request)`. Store the returned `order_id`.
5. Start a staleness timer: `asyncio.create_task(self._stale_check(order_id, pair_name))`.

11.4.2 On Exit

1. Cancel any open entry orders for this pair.
2. If a position is open, place a market sell/buy (taker) to close immediately.
3. Record the `TradeRecord` in the database.

11.4.3 On Reversal

Execute Exit followed by Entry in the opposite direction.

11.4.4 On Scale

If the current position size is less than the new target size, place an additional limit order for the difference.

11.4.5 On Reduce

If `reduce_on_conviction_drop` is enabled in config, place a reduce-only limit order to trim the position to the new target size.

11.5 Staleness Check

```
async def _stale_check(
    self, order_id: str, pair_name: str
) -> None:
    """Sleep for stale_order_seconds. If order is still open
       (not filled), cancel it. Log as 'stale_cancel'."""
    await asyncio.sleep(self.config.execution.stale_order_seconds)
    order = self.open_orders.get(order_id)
    if order and order.status in (OrderStatus.PLACED, OrderStatus.
       PENDING):
        await self.deribit.cancel_order(order_id)
        order.status = OrderStatus.CANCELLED
        order.cancel_reason = "stale"
        await self.database.update_order(order_id, status="cancelled"
           ,
                                         cancel_reason="stale")
```

11.6 Order Update Handler

Register as a callback on `deribit.subscribe_orders`. When an order status changes:

(a) **Filled:** Update `OrderState`. Create or update `Position`. Persist to database.

(b) **Partially filled:** Update `filled_size`. Treat as a full entry at the filled portion. Cancel the remainder.

(c) **Cancelled/Rejected:** Update `OrderState`. Log the reason. If the cancellation was not initiated by us (e.g. Deribit self-trade prevention), log a warning.

11.7 Forced Exit Handler

Register as a callback from the Risk Manager's `set_on_exit`. When called:

1. Cancel all open orders for the pair.
2. Place a market order to close the position.
3. Record `TradeRecord` with the appropriate `exit_reason`.

11.8 Class Interface

```python
class ExecutionManager:
    def __init__(
        self,
        config: Config,
        database: Database,
        deribit: DeribitClient,
        risk_manager: RiskManager
    ): ...

    async def on_signal(self, state: SignalState) -> None:
        """Main entry point from signal engine.
           Dispatch to transition handler."""

    async def on_forced_exit(
        self, pair_name: str, reason: str
    ) -> None:
        """Called by risk manager for stop/TP exits."""

    async def _handle_entry(
        self, pair: str, state: SignalState
    ) -> None: ...

    async def _handle_exit(
        self, pair: str, state: SignalState
    ) -> None: ...

    async def _handle_reversal(
        self, pair: str, state: SignalState
    ) -> None: ...

    async def _handle_scale(
        self, pair: str, state: SignalState
    ) -> None: ...

    async def _handle_reduce(
        self, pair: str, state: SignalState
    ) -> None: ...

    async def _stale_check(
        self, order_id: str, pair_name: str
    ) -> None: ...

    async def _on_order_update(self, data: dict) -> None:
        """Callback for Deribit order subscription."""

    async def _on_trade_update(self, data: dict) -> None:
        """Callback for Deribit trade subscription."""

    def get_position(self, pair_name: str) -> Optional[Position]: ...
    def get_open_orders(self, pair_name: str) -> list[OrderState]:
        ...
```

12 Main Entry Point

```python
# main.py
```

```python
import asyncio
from config import load_config
from database import Database
from logger import setup_logger
from event_listener import EventListener
from signal_engine import SignalEngine
from deribit_client import DeribitClient
from execution_manager import ExecutionManager
from risk_manager import RiskManager

async def main():
    config = load_config("config.yaml")
    logger = setup_logger(config.system.log_level)
    logger.info("Starting momentum system", mode=config.system.mode)

    # Initialize components
    db = Database(config.system.database_path)
    await db.initialize()

    deribit = DeribitClient(config.deribit)
    await deribit.connect()

    risk_mgr = RiskManager(config, db, deribit)
    exec_mgr = ExecutionManager(config, db, deribit, risk_mgr)
    signal_eng = SignalEngine(config, db, on_signal=exec_mgr.
        on_signal)
    listener = EventListener(config, db, on_event=signal_eng.on_event
        )

    # Wire risk manager exit callback
    risk_mgr.set_on_exit(exec_mgr.on_forced_exit)

    # Subscribe to Deribit channels for each instrument
    instruments = set(p.deribit_instrument for p in config.pairs)
    for inst in instruments:
        currency = inst.split("-")[0]
        await deribit.subscribe_book(inst, exec_mgr._on_book_update)
        await deribit.subscribe_orders(inst, exec_mgr.
            _on_order_update)
        await deribit.subscribe_trades(inst, exec_mgr.
            _on_trade_update)
        await deribit.subscribe_portfolio(
            currency, risk_mgr._on_portfolio_update
        )

    # Start all tasks
    await asyncio.gather(
        listener.start(),
        risk_mgr.start_monitor(),
        _heartbeat(config.system.heartbeat_interval_seconds, logger,
                   signal_eng, exec_mgr, risk_mgr),
    )

async def _heartbeat(interval, logger, signal_eng, exec_mgr, risk_mgr
    ):
    """Periodic status log."""
    while True:
        await asyncio.sleep(interval)
```

```
        for pair in signal_eng._states:
            state = signal_eng.get_state(pair)
            pos = exec_mgr.get_position(pair)
            logger.info("heartbeat",
                pair=pair,
                signal=state.previous_signal,
                trend=state.trend_state_30,
                conviction=round(state.conviction_30, 3),
                position=pos.size if pos else 0,
                direction=pos.direction.name if pos else "NONE",
            )

if __name__ == "__main__":
    asyncio.run(main())
```

13 Backtesting Framework

13.1 Purpose

Replay historical swap events from the database through the Signal Engine, simulate order placement and fills against historical Deribit prices (or proxied by the 1 bps pool price), and compute performance metrics.

13.2 Architecture

The backtester reuses `SignalEngine` directly. It replaces `DeribitClient` and `ExecutionManager` with simulated versions.

```
class BacktestEngine:
    def __init__(
        self,
        config: Config,
        database: Database,
        execution_lag_blocks: int = 1,
    ): ...

    async def run(
        self,
        pair_name: str,
        start_timestamp: int,
        end_timestamp: int,
    ) -> BacktestResult: ...
```

13.3 Simulated Fill Logic

When the signal engine emits an entry signal at time t_{sig}:

1. Look up the 1 bps pool price at t_{sig}. This proxies the Deribit mid price.
2. Compute the limit price using the same offset formula from Section 11.
3. Scan forward through 1 bps events within `stale_order_seconds` of t_{sig}.
4. If any event has a price that crosses the limit price (i.e. price $\leq$ limit for a long, or price $\geq$ limit for a short), record a fill at the limit price at that timestamp.

5. If no crossing event occurs within the staleness window, record a missed signal.

For exits, simulate market fills at the 1 bps pool price at the exit timestamp plus `execution_lag_blocks` blocks of delay.

13.4 BacktestResult

```
class BacktestResult(BaseModel):
    pair_name: str
    start_time: float
    end_time: float
    total_signals: int
    filled_signals: int
    fill_rate: float                # filled / total
    total_trades: int
    win_rate: float
    avg_gross_return_bps: float
    avg_net_return_bps: float       # after estimated taker fee on
        stops
    total_pnl_usd: float
    max_drawdown_usd: float
    max_drawdown_bps: float
    sharpe_ratio: float             # annualized, on daily returns
    avg_holding_seconds: float
    trades: list[TradeRecord]
    equity_curve: list[tuple[float, float]]  # (timestamp,
        cumulative_pnl)
```

13.5 Alpha Decay Curve

For each 30 bps event, measure the 1 bps pool price change at 1, 2, 5, 10, 20, 50, and 100 subsequent events after the trigger:

$$\text{continuation}_n = \text{sign}(r_k^{30}) \times \left(\ln P_{t+n}^1 - \ln P_{t_{\text{trigger}}}^1\right) \times 10000 \text{ (bps)} \tag{18}$$

Average across all 30 bps events to produce the alpha decay curve. If the curve flattens before $n = 5$ events, the edge is likely too fast to capture.

13.6 Running the Backtest

```
# backtest.py (entry point when run directly)
async def main():
    config = load_config("config.yaml")
    db = Database(config.system.database_path)
    await db.initialize()

    engine = BacktestEngine(config, db, execution_lag_blocks=1)
    result = await engine.run(
        pair_name="ETH-USDC",
        start_timestamp=1700000000,
        end_timestamp=1710000000,
    )

    print(f"Signals: {result.total_signals}")
```

```
    print(f"Fill rate: {result.fill_rate:.1%}")
    print(f"Win rate: {result.win_rate:.1%}")
    print(f"Avg net return: {result.avg_net_return_bps:.1f} bps")
    print(f"Sharpe: {result.sharpe_ratio:.2f}")
    print(f"Max DD: {result.max_drawdown_usd:.2f} USD")

if __name__ == "__main__":
    asyncio.run(main())
```

14 Logging

Use Python's built-in `logging` module. Configure structured JSON output for production, human-readable for development.

```
# logger.py
import logging
import json
import sys

def setup_logger(level: str = "INFO") -> logging.Logger:
    logger = logging.getLogger("momentum")
    logger.setLevel(getattr(logging, level.upper()))

    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JsonFormatter())
    logger.addHandler(handler)
    return logger

class JsonFormatter(logging.Formatter):
    def format(self, record):
        log_data = {
            "ts": record.created,
            "level": record.levelname,
            "msg": record.getMessage(),
        }
        # Merge any extra kwargs passed via logger.info("msg", extra
            ={...})
        if hasattr(record, "_extra"):
            log_data.update(record._extra)
        return json.dumps(log_data)
```

Every log message should include the component name and pair name where applicable. Log at the following levels:

- `DEBUG`: Every swap event received, every signal recomputation.
- `INFO`: Signal transitions, order placements, fills, cancellations, heartbeats.
- `WARNING`: WebSocket disconnections, order rejections by risk manager, staleness cancellations.
- `ERROR`: Failed order submissions, database errors, authentication failures.

15 Testing Requirements

15.1 test_signal_engine.py

T1. Feed a sequence of 30 bps events all in the same direction. Assert conviction increases and trend state transitions to +1 after crossing the entry threshold.

T2. Feed alternating-direction 30 bps events. Assert conviction stays near zero and trend state stays at 0.

T3. Feed 30 bps events to establish a trend, then feed 5 bps events in the same direction with positive autocorrelation. Assert that $F^5 = 1$ and $S \neq 0$.

T4. Feed 5 bps events with negative autocorrelation. Assert $F^5 = 0$ regardless of 30 bps trend.

T5. Test hysteresis: establish trend at +1, then feed weak opposing events that bring conviction between exit and entry thresholds. Assert trend remains +1.

T6. Test transition detection for all five types: entry, exit, scale, reduce, reversal.

T7. Test conviction decay over time: feed an event, wait (simulate passage of time), feed another. Assert conviction has decayed.

15.2 test_execution_manager.py

T1. Mock `DeribitClient`. Emit an entry signal. Assert `place_order` is called with the correct direction, size, and limit price (within tick-size rounding).

T2. Emit an entry signal, then an exit signal. Assert the open order is cancelled and a market close order is placed.

T3. Emit an entry signal, simulate a fill, then emit a reversal. Assert close followed by new entry in opposite direction.

T4. Emit an entry signal, wait past `stale_order_seconds`, assert the order is cancelled.

T5. Emit an entry signal with $|S_t| = 0.3$. Assert size is approximately $0.3\times$`max_position_contracts`.

15.3 test_risk_manager.py

T1. Attempt to place an order during cooldown. Assert rejection.

T2. Attempt to place an order exceeding `max_position_contracts`. Assert rejection.

T3. Simulate cumulative daily losses exceeding `daily_loss_limit_usd`. Assert rejection.

T4. Simulate a position with unrealized loss exceeding `stop_loss_bps`. Assert the exit callback is invoked.

T5. Simulate a position with unrealized profit exceeding `take_profit_bps`. Assert the take-profit logic is invoked.

16 Deployment Phases

Phase 1: Data Collection (2 weeks). Run with `mode: backtest` collection—deploy only the Event Listener and Database. Collect swap events from all pools. Validate data: check for gaps, confirm prices are monotonically sensible, confirm that 30 bps events are sparser than 5 bps events.

Phase 2: Backtest (1 week). Run the backtesting framework. Sweep parameters. Compute the alpha decay curve. If the curve shows no continuation beyond 2–3 blocks at realistic latency, **stop here**.

Phase 3: Paper Trading (2 weeks). Run with `mode: paper`. The full pipeline runs but orders are simulated. Compare simulated fills to actual Deribit prices at the timestamps where orders would have been placed.

Phase 4: Live—Small (4 weeks). Run with `mode: live` at 10% of target position sizes, on Deribit **testnet** first, then mainnet. Monitor fill rates, slippage, and realized PnL versus paper expectations.

Phase 5: Live—Full (ongoing). Scale to target sizes. Continue monitoring all metrics.

17 Kill Criteria

Shut down the system if any of the following are observed over a rolling window:

(1) Average net return per trade < 3 bps over the last 100 trades.

(2) Rolling 30-day annualized Sharpe ratio < 0.5.

(3) Fill rate drops below 15% over the last 200 signals.

(4) The alpha decay curve, re-estimated on live data, shows >70% of continuation captured within 1 block of the 30 bps event.

End of specification.